• PROGRAMACIÓN AVANZADA · UTU · BACHILLERATO TECNOLÓGICO

Actividades de práctica — Base de Datos

Tres actividades para consolidar la conexión a MySQL con JDBC y las 5 fases del acceso a datos aplicadas a consultas: primero sobre la tabla personas y después sobre la entidad propia de la Práctica 2.

TEMA 3 — JDBC + MYSQL | SOLO CONSULTAS (SELECT) | ACT. 03 — ENTIDAD PROPIA | DIFICULTAD CRECIENTE

ALCANCE DE ESTA PRÁCTICA

En esta práctica el DAO se usa solo para **traer datos** de la base, con consultas `SELECT`. Los datos se cargan directamente en MySQL Workbench con sentencias `INSERT` y `UPDATE`. Los métodos del DAO para agregar, actualizar y eliminar desde la aplicación se trabajarán en las próximas clases.

01

CONSOLIDACIÓN · TEMA 3 — JDBC

Mostrar la fecha_nacimiento de cada persona

NIVEL BÁSICO

Agregar a la tabla **personas** la columna **fecha_nacimiento** (tipo DATE), cargar fechas directamente en MySQL y mostrarlas en el listado de la aplicación. El cambio recorre las capas que intervienen en una consulta: la tabla, el VO, el método `getLista()` del DAO y la vista `persona-lista.jsp`.

PROYECTO BASE	SQL NUEVO
`mini-crud-sql-base`	**ALTER TABLE · DATE**

// QUÉ CAMBIA EN CADA CAPA

CAPA	ARCHIVO	CAMBIO
Base de datos	`tabla personas`	Agregar la columna con `ALTER TABLE` y cargar fechas con `UPDATE` e `INSERT`
Modelo · VO	`PersonaVO.java`	Atributo `fechaNacimiento`, constructor de 3 parámetros, getter y setter
Modelo · DAO	`getLista()`	Fase 5: leer la columna con `rs.getDate("fecha_nacimiento")`
Vista	`persona-lista.jsp`	Columna nueva con `${p.fechaNacimiento}`

// TAREAS

1. En **MySQL Workbench**, agregar la columna, cargar fechas y comprobar el resultado con el script siguiente. La persona con código 3 queda a propósito sin fecha.

MySQL Workbench — columna nueva y datos

```sql
USE institutoWeb;

-- 1. Agregar la columna sin perder los datos que ya existen
ALTER TABLE personas ADD COLUMN fecha_nacimiento DATE;

-- 2. Cargar fechas a personas que ya están en la tabla
UPDATE personas SET fecha_nacimiento = '2008-03-15' WHERE codigo = 1;
UPDATE personas SET fecha_nacimiento = '2007-11-02' WHERE codigo = 2;

-- 3. Agregar una persona nueva, ya con su fecha
INSERT INTO personas (codigo, nombre, fecha_nacimiento)
VALUES (4, 'Lucía', '2008-07-21');

-- 4. Comprobar el resultado
SELECT * FROM personas;
```

2. En **PersonaVO.java**: declarar `private java.sql.Date fechaNacimiento`. Agregar un **segundo constructor** que reciba código, nombre y fecha, sin borrar el de 2 parámetros, que el Servlet y otros métodos del DAO siguen usando. Agregar getter y setter.

3. En **PersonaDAO.java**: modificar solo `getLista()`. La consulta `SELECT * FROM personas` ya trae la columna nueva; en la fase 5 leerla con `rs.getDate("fecha_nacimiento")` y usar el constructor de 3 parámetros. Los demás métodos no se tocan.

4. En **persona-lista.jsp**: agregar `<th>Fecha de nacimiento</th>` en el encabezado y la celda `${p.fechaNacimiento}` dentro del `<c:forEach>`. Actualizar el `colspan` de la fila "No hay personas registradas" de 3 a 4.

// REFERENCIAS DE CÓDIGO

PersonaVO.java — atributo y constructor nuevos

```java
private java.sql.Date fechaNacimiento;      // ← nuevo atributo

// Se mantiene el constructor de 2 parámetros: el Servlet y otros métodos del DAO lo usan
public PersonaVO(int cod, String nombre) { ... }

// Constructor nuevo, con la fecha (3 parámetros)
public PersonaVO(int cod, String nombre, java.sql.Date fechaNacimiento) {
    this.cod = cod;
    this.nombre = nombre;
    this.fechaNacimiento = fechaNacimiento;
}

public java.sql.Date getFechaNacimiento() { return fechaNacimiento; }
public void setFechaNacimiento(java.sql.Date f) { this.fechaNacimiento = f; }
```

PersonaDAO.java — fase 5 de getLista()

```java
// FASE 5: Leer el ResultSet
while (rs.next()) {
    lista.add(new PersonaVO(
            rs.getInt("codigo"),
            rs.getString("nombre"),
            rs.getDate("fecha_nacimiento")));   // ← nuevo
}
```

persona-lista.jsp — columna nueva

```jsp
<!-- En el <thead> -->
<th>Código</th>
<th>Nombre</th>
<th>Fecha de nacimiento</th>
<th>Acciones</th>

<!-- Dentro del <c:forEach var="p" ...> -->
<td>${p.codigo}</td>
<td>${p.nombre}</td>
<td>${p.fechaNacimiento}</td>
```

¿Y SI UNA PERSONA NO TIENE FECHA?

Si en MySQL la columna quedó en `NULL`, `rs.getDate()` devuelve `null` y la celda se muestra vacía. Es el comportamiento esperado. Lo mismo pasa con las personas que se agreguen desde el formulario de la aplicación: `agregar()` todavía no guarda la fecha.

// CRITERIOS DE VERIFICACIÓN

- [ ] En MySQL, `SELECT * FROM personas` muestra la columna `fecha_nacimiento` con las fechas cargadas.
- [ ] El listado de la aplicación muestra la columna Fecha de nacimiento con los mismos valores que MySQL.
- [ ] La persona sin fecha aparece en el listado con la celda vacía, sin errores.
- [ ] El resto de la aplicación (agregar, editar, eliminar) sigue funcionando como antes.

02

APLICACIÓN · TEMA 3 — CONSULTA PERSONALIZADA

Búsqueda de personas por nombre en la BD

NIVEL INTERMEDIO

Agregar un método nuevo al DAO que traiga de la base **solo las personas cuyo nombre contiene un texto**, usando SQL `LIKE`. Integrarlo con una acción nueva en el Servlet y un formulario de búsqueda en la vista del listado, de modo que la tabla muestre solo los resultados que coinciden.

PROYECTO BASE	SQL NUEVO
mini-crud-sql-base	LIKE con % (comodín)

// PREPARAR LOS DATOS EN MYSQL

MySQL Workbench — datos de prueba

```sql
-- Cargar algunas personas más para probar la búsqueda
INSERT INTO personas (codigo, nombre)
VALUES (5, 'Ana'), (6, 'Mariana'), (7, 'Fabiana'), (8, 'Diego');
```

Si se hizo la Actividad 01, estas personas quedan sin fecha de nacimiento; no es un problema. Recordar que `nombre` admite hasta 10 caracteres.

// CONCEPTO: SQL LIKE

SQL — búsqueda parcial con LIKE

```sql
-- El % actúa como comodín: cualquier cantidad de caracteres
SELECT * FROM personas WHERE nombre LIKE '%ana%';
-- devuelve: Ana, Mariana, Fabiana
```

// TAREAS

1. En **PersonaDAO.java**: agregar el método `buscarPorNombre(String termino)` que devuelva `List<PersonaVO>`. El SQL usa `LIKE ?` y el parámetro es `"%" + termino + "%"`, para buscar en cualquier parte del nombre. Aplicar las 5 fases, tomando como modelo `getLista()` y abriendo el `ResultSet` en un try anidado, después de asignar el parámetro.

2. En **PersonaControladorServlet.java**, en el `doGet`: agregar un `case "buscar"` que lea el parámetro `termino`, llame a `dao.buscarPorNombre(termino)`, guarde el resultado con `request.setAttribute("personas", ...)` y haga `forward` a `/WEB-INF/vista/persona-lista.jsp`.

3. En **persona-lista.jsp**: agregar un formulario de búsqueda encima de la tabla, con un campo de texto (`name="termino"`), un botón "Buscar" y un enlace "Ver todos". El formulario envía por `GET` con `action=buscar`.

4. Si la búsqueda no devuelve resultados, mostrar el mensaje "No se encontraron personas con ese nombre." en lugar de "No hay personas registradas". Pista: `${param.termino}` indica si se está buscando.

// REFERENCIAS DE CÓDIGO

PersonaDAO.java — estructura de buscarPorNombre()

```java
public List<PersonaVO> buscarPorNombre(String termino) {
    List<PersonaVO> lista = new ArrayList<>();
    // FASE 1: Definir la sentencia SQL
    String sql = "SELECT * FROM personas WHERE nombre LIKE ?";

    // FASES 2 y 3: conexión y PreparedStatement
    try (Connection con = Conexion.obtener();
         PreparedStatement ps = con.prepareStatement(sql)) {

        // FASE 3: asignar el parámetro (el % va en el valor, no en el SQL)
        ps.setString(1, "%" + termino + "%");

        // FASE 4: Ejecutar
        try (ResultSet rs = ps.executeQuery()) {
            // FASE 5: recorrer el ResultSet igual que en getLista()
        }
    } catch (SQLException e) {
        System.err.println("Error al buscar personas: " + e.getMessage());
    }
    return lista;
}
```

persona-lista.jsp — formulario de búsqueda

```jsp
<form action="${pageContext.request.contextPath}/persona" method="get">
    <input type="hidden" name="action" value="buscar">
    <input type="text" name="termino" value="${param.termino}">
    <button class="btn" type="submit">Buscar</button>
    <a class="btn btn-secondary" href="${pageContext.request.contextPath}/persona">Ver todos</a>
</form>
```

POR QUÉ GET Y NO POST PARA LA BÚSQUEDA

Las búsquedas usan `method="get"` porque no modifican datos. El término buscado queda en la URL (`?action=buscar&termino=ana`), lo que permite compartir o guardar el enlace de los resultados.

// CRITERIOS DE VERIFICACIÓN

- [ ] Buscar "an" muestra Ana, Mariana y Fabiana, y ninguna otra persona.
- [ ] La búsqueda funciona si el texto está al inicio, en el medio o al final del nombre.
- [ ] Si no hay coincidencias, se muestra "No se encontraron personas con ese nombre." en lugar de una tabla vacía sin explicación.
- [ ] El enlace "Ver todos" vuelve al listado completo, sin filtro.

03

DESAFÍO · TEMA 3 — DE MEMORIA A MYSQL

Llevar la entidad propia a MySQL

NIVEL AVANZADO

Retomar el proyecto de la **Actividad 03 de la Práctica 2** (el CRUD de la entidad elegida: Producto, Libro, Película, Videojuego, Estudiante u otra) y hacer que su **listado salga de MySQL** en lugar del `ArrayList` en memoria. Como por ahora el DAO solo trae datos, los botones Agregar, Editar y Eliminar quedan deshabilitados hasta las próximas clases. Los ejemplos usan **Producto**: reemplazar por la entidad propia.

PROYECTO BASE	REFERENCIA
Act. 03 · Práctica 2	mini-crud-sql-base · getLista()

// TIPOS DE JAVA A MYSQL

JAVA (VO)	MYSQL (TABLA)	LECTURA EN LA FASE 5
int	`INT`	`rs.getInt("columna")`
String	`VARCHAR(n)`	`rs.getString("columna")`
double	`DECIMAL(8,2)` o `DOUBLE`	`rs.getDouble("columna")`

// TAREAS

1. En **MySQL Workbench**: crear la tabla de la entidad, con la clave primaria en el código, y cargar al menos 4 registros con `INSERT`. Guardar el script como `entidad.sql` para la entrega.

MySQL Workbench — tabla y datos (ejemplo con Producto)

```
USE institutoWeb;

CREATE TABLE productos (
  codigo       INT PRIMARY KEY,
  nombre       VARCHAR(30) NOT NULL,
  precio       DECIMAL(8,2) NOT NULL,
  descripcion  VARCHAR(100)
);

INSERT INTO productos (codigo, nombre, precio, descripcion)
VALUES (1, 'Teclado', 850.00, 'Teclado USB'),
       (2, 'Mouse', 420.50, 'Mouse óptico'),
       (3, 'Monitor', 6900.00, 'Monitor de 22 pulgadas'),
       (4, 'Auriculares', 1290.00, 'Con micrófono');

SELECT * FROM productos;
```

2. Conectar el proyecto a la base: agregar al `pom.xml` la dependencia `com.mysql:mysql-connector-j`, y copiar de `mini-crud-sql-base` la clase `modelo/conexion/Conexion.java` y el archivo `src/main/resources/db.properties`. Revisar que el usuario y la contraseña de `db.properties` sean los de la propia máquina.

3. En **ProductoDAO.java**: reescribir por dentro `getLista()`, que devuelve `ArrayList<ProductoVO>` y aplica las 5 fases con `SELECT * FROM productos`, tomando como modelo `getLista()`. Mantiene su nombre, así que el Servlet no cambia. Los demás métodos siguen con el `ArrayList`.

4 En **ProductoControladorServlet.java**: no cambia. El método `listar()` sigue usando `dao.getLista()`, ya que `getLista()` no cambió de nombre.

5 En **producto-lista.jsp**: reemplazar los botones Editar y Eliminar de cada fila, y el botón Agregar, por botones deshabilitados con la clase `btn-disabled`. Si el `styles.css` del proyecto no tiene esa clase, agregarla.

// REFERENCIAS DE CÓDIGO

ProductoDAO.java — estructura de getLista()

```java
public ArrayList<ProductoVO> getLista() {
    ArrayList<ProductoVO> lista = new ArrayList<>();
    // FASE 1: Definir la sentencia SQL
    String sql = "SELECT * FROM productos";

    // FASES 2, 3 y 4: conexión, PreparedStatement y ejecución
    try (Connection con = Conexion.obtener();
         PreparedStatement ps = con.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {

        // FASE 5: Leer el ResultSet
        while (rs.next()) {
            // crear un ProductoVO con rs.getInt(...), rs.getString(...),
            // rs.getDouble(...) y agregarlo a la lista
        }
    } catch (SQLException e) {
        System.err.println("Error al listar productos: " + e.getMessage());
    }
    return lista;
}
```

ProductoControladorServlet.java — listar() sin cambios

```java
private void listar(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    request.setAttribute("productos", dao.getLista());   // igual que antes: no cambia
    request.getRequestDispatcher("/WEB-INF/vista/producto-lista.jsp")
           .forward(request, response);
}
```

producto-lista.jsp — botones deshabilitados

```html
<td class="actions">
    <button disabled class="btn-disabled" title="Funcionalidad no disponible">Editar</button>
    <button disabled class="btn-disabled" title="Funcionalidad no disponible">Eliminar</button>
</td>
```

// ARCHIVOS A CREAR Y MODIFICAR

Estructura esperada (ejemplo con Producto)

```
Base de datos:
  tabla productos → CREATE TABLE + INSERT desde Workbench

Configuración:
  pom.xml   ← dependencia mysql-connector-j
  src/main/resources/db.properties   ← copiar de mini-crud-sql-base
  src/main/java/modelo/conexion/Conexion.java   ← copiar de mini-crud-sql-base

Capa Modelo:
  src/main/java/modelo/dao/ProductoDAO.java   ← getLista() lee de la BD

Capa Controlador:
  src/main/java/controlador/ProductoControladorServlet.java   ← sin cambios

Capa Vista:
  src/main/webapp/WEB-INF/vista/producto-lista.jsp   ← botones deshabilitados
  src/main/webapp/css/styles.css   ← clase .btn-disabled (si falta)
```

> **QUÉ CAMBIA Y QUÉ NO**
>
> Pasar de memoria a MySQL cambia **de dónde vienen los datos**: la tabla, la conexión y el método del DAO. El VO y la vista del listado quedan casi iguales. Esa es una de las ventajas de separar el proyecto en capas con MVC.

// CRITERIOS DE VERIFICACIÓN

- [ ] En MySQL, `SELECT * FROM` la tabla de la entidad devuelve al menos 4 registros.
- [ ] El listado de la aplicación muestra exactamente los mismos registros que MySQL.
- [ ] Al agregar una fila nueva con `INSERT` en Workbench y recargar la página, la fila aparece sin volver a compilar.
- [ ] Los botones Agregar, Editar y Eliminar se ven en gris y no realizan ninguna acción.
- [ ] La entrega incluye el script `entidad.sql` con el `CREATE TABLE` y los `INSERT`.